

Multimodal Retrieval-Augmented Generation for the Sri Lanka Tourism Sector

SCS 4203, Database III, Assignment 2

Student 1	<i>[Name], [Index No]</i>
Student 2	<i>[Name], [Index No]</i>
Date	<i>[Submission date]</i>

1 Introduction

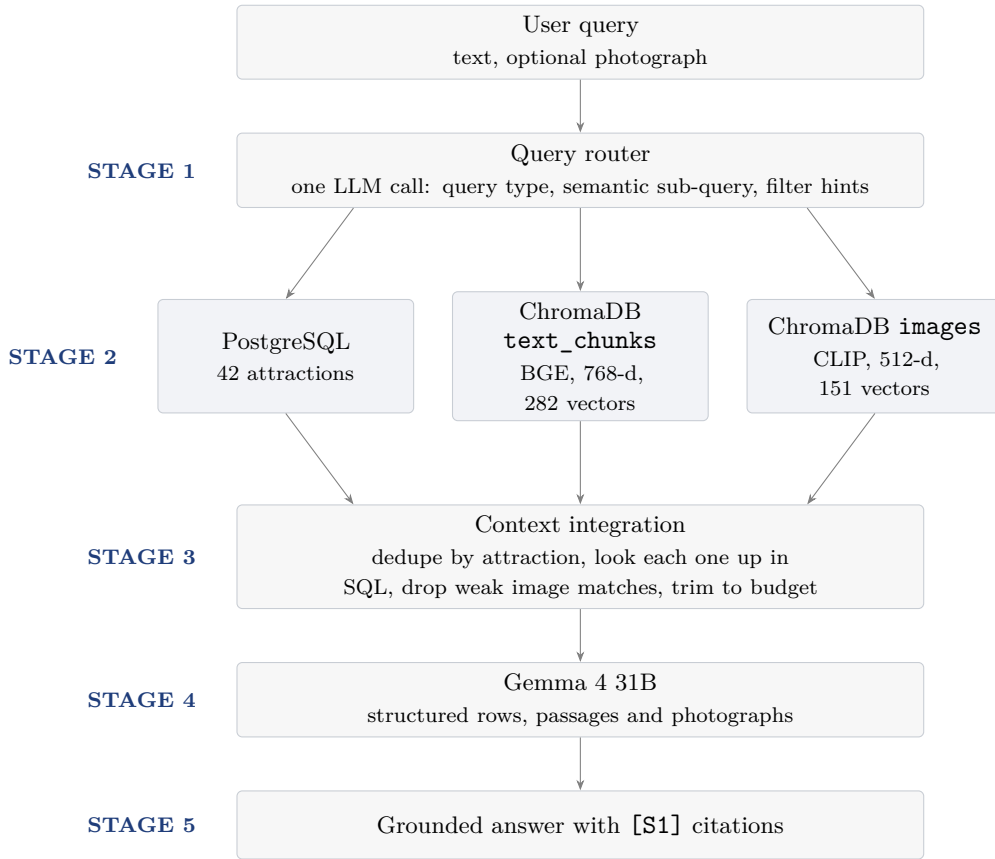
Tourists ask questions of several different kinds. “Which waterfalls are over 100 metres tall?” filters on a numeric attribute. “Where can I see colonial-era fortifications beside the sea?” names no attraction and no column, so it has to be matched on meaning. “What is this place?” asked over a photograph is neither of those.

Each kind suits a different store. A relational database answers exactly and can count and sort, but cannot match on meaning. A text-embedding index matches on meaning but cannot filter reliably, and will return a passage that sounds right while containing the wrong number. An image-embedding index handles visual similarity, which neither of the others can express.

This system uses all three, so the design problem is how to combine them: what decides which stores to consult, and how their outputs become one context for the language model. Section 6 covers that, and Section 7 measures whether combining them helps.

The knowledge base covers waterfalls, beaches, and historical sites and temples. The brief allows a subset of categories, and these three exercise different parts of the schema: waterfalls have numeric attributes, heritage sites have categorical and boolean ones, and beaches are described mostly in prose.

2 System architecture



Stage 3 combines the three result sets. Section 6.4 describes it.

3 Knowledge base

The knowledge base holds 42 attractions across 19 districts, 14 in each category, with 282 text chunks and 164 photographs.

Asset	Count
Text chunks from Wikipedia	205
Text chunks from AmazingLanka	77
Photographs indexed	151
Photographs held out for evaluation	13

3.1 Two text sources

The first version used Wikipedia alone, which turned out to cover the three categories very unevenly. The Sigiriya article runs to 2,660 words. Bambarakanda Falls, the tallest waterfall in Sri Lanka, is a 131-word stub, and several other waterfalls are shorter. Semantic search over that corpus would have worked well for heritage sites and poorly for waterfalls.

AmazingLanka (amazinglanka.com), a Sri Lankan travel encyclopaedia, covers the attractions Wikipedia treats thinly, including access routes and trail conditions. It runs on WordPress, so we read it through its public REST API rather than scraping HTML. It supplied text for 26 of the 42 attractions and took Bambarakanda Falls from 78 words to roughly 350.

Its search is fuzzy: querying “Bambarakanda” returns an article about the Devil’s Staircase trail among the top results. A candidate article is therefore accepted only if its title is similar enough to the attraction name, since attaching the wrong article is worse than attaching none.

3.2 Licensing

Wikipedia and Wikimedia Commons are CC BY-SA, so both text and photographs are used, with the licence and author of each image stored in the `images` table. AmazingLanka’s content is copyrighted, so we use its text for non-commercial academic purposes with per-chunk attribution and take none of its photographs. All 164 images come from Commons.

Every row in `documents` records its `source` and `source_url`, so any retrieved passage can be traced back to its page, and every row in `images` records its licence and author.

3.3 Curation

Neither source reliably gives entrance fees, trekking difficulty or dress codes. Those were entered by hand in `data/seeds/attractions.csv`, which the ingest script treats as authoritative for structured fields while the web sources supply prose and images.

4 Database design

4.1 Relational schema

```
attractions (
  id SERIAL PRIMARY KEY,
  name TEXT NOT NULL UNIQUE,
  category TEXT NOT NULL CHECK (category IN ('waterfall','beach','heritage')),
  district TEXT, province TEXT, latitude FLOAT8, longitude FLOAT8,
  height_m NUMERIC(6,1),          -- waterfalls
  trekking_difficulty TEXT,       -- waterfalls
  swimming_safety TEXT,          -- beaches
  surf_season TEXT,              -- beaches
  era TEXT,                      -- heritage
  unesco_status BOOLEAN,         -- heritage
  dress_code TEXT,               -- heritage
  entrance_fee_lkr INT, best_season TEXT, accessibility TEXT,
  summary TEXT, wiki_url TEXT
)

images (
  id SERIAL PRIMARY KEY,
  attraction_id INT NOT NULL REFERENCES attractions(id) ON DELETE CASCADE,
  file_path TEXT NOT NULL, caption TEXT,
  source_url TEXT, license TEXT, is_held_out BOOLEAN DEFAULT FALSE
)

documents (
  id SERIAL PRIMARY KEY,
  attraction_id INT NOT NULL REFERENCES attractions(id) ON DELETE CASCADE,
  chunk_index INT NOT NULL, chunk_text TEXT NOT NULL,
  source TEXT CHECK (source IN ('wikipedia','amazinglanka')), source_url TEXT,
  UNIQUE (attraction_id, chunk_index)
)
```

Indexed on `category`, `district`, `height_m` and `entrance_fee_lkr`, plus `attraction_id` on both child tables.

4.2 Design decisions

One wide table instead of per-category subtypes. The normalised design would put `height_m` in a `waterfalls` table and `dress_code` in a `heritage_sites` table, both joined to a common supertype.

That is cleaner, because it makes it structurally impossible to set a waterfall’s height on a beach. We rejected it for a reason specific to this system. The SQL retriever is a language model writing queries from a schema in its prompt (Section 6.2). Every extra table is another join it has to get right. With one table the generated SQL is a single `SELECT`, which is more reliable. The cost is that most category-specific columns are `NULL` on any given row: 28 of 42 rows have no `height_m`, correctly, because only the 14 waterfalls have a height. This is the decision we would most want to revisit.

unesco_status marks inscribed sites only. Six attractions are UNESCO World Heritage Sites. Ruwanwelisaya, Jetavanaramaya and Isurumuniya stand inside the Sacred City of Anuradhapura, which is itself one inscribed site, so flagging them separately would make “how many UNESCO sites are there?” answer nine and count one site four times. The column means that the row is itself an inscribed site.

documents holds the authoritative text. ChromaDB stores only vectors, ids and filter metadata. The whole vector index can be rebuilt from PostgreSQL by re-running `embed.py`, so the two stores cannot drift apart.

4.3 Vector store

Collection	Encoder	Dim	Vectors
text_chunks	BAAI/bge-base-en-v1.5	768	282
images	clip-ViT-B-16	512	151

Both use cosine distance. `text_chunks` carries `attraction_id`, `name`, `category`, `district` and the source fields as metadata; `images` carries the same minus the source fields, plus `file_path`.

The two collections are separate because the encoders produce vectors in unrelated spaces. Comparing a BGE vector to a CLIP vector raises no error, it returns meaningless neighbours, so separate collections make that mistake impossible to write.

5 Embedding generation

Both encoders run locally, so ingestion and indexing need no API access. Only routing, SQL generation and answering call an external model.

5.1 Text

Articles are split into overlapping windows of 220 words with 50 words of overlap, and each chunk is prefixed with its attraction’s name before encoding. Without it, a chunk reading “It is 263 m high and has a single drop” tells neither the embedding nor the model which attraction “it” is.

Wikipedia’s trailing sections (*See also*, *References*, *External links*) are stripped first, as are AmazingLanka’s navigation blocks and its star-rating widget, which renders as text and would otherwise be embedded along with the article. Vectors are L2-normalised, so cosine distance matches the dot product. The result is 282 vectors of 768 dimensions.

5.2 Images

Photographs are converted to RGB, resized so the longest edge is 512px, and encoded with CLIP ViT-B/16 into 512 dimensions. CLIP resizes to 224px internally, so the stored 512px is for display in the interface rather than for embedding quality.

CLIP was chosen over an image-only encoder because it puts images and text in one shared

space, which gives two capabilities from one index: image-to-image search when a photograph is uploaded, and text-to-image search when someone types “a wide waterfall over a rock face” instead. Both hit the same collection.

Thirteen photographs were kept out of the index. The ingest script marks the fourth photograph of every waterfall `is_held_out` and the embedding script skips those rows, so the system can be tested on images it has not seen.

5.3 Choosing the encoders

Both encoders were selected by measurement rather than reputation. Each candidate was scored on the same data: text encoders on the 22 semantic and hybrid evaluation questions, image encoders on the 13 held-out photographs.

Text encoder	dim	R@1	R@3	MRR
bge-base-en-v1.5	768	68.2%	90.9%	0.792
bge-small-en-v1.5	384	68.2%	81.8%	0.761
nomic-embed-text-v1.5	768	63.6%	86.4%	0.746
bge-small-en-v1.5 + query prefix	384	63.6%	77.3%	0.739
e5-base-v2	768	59.1%	77.3%	0.689

Image encoder	dim	top-1	top-3	MRR
clip-ViT-B-16	512	61.5%	69.2%	0.673
clip-ViT-L-14	768	61.5%	61.5%	0.646
clip-ViT-B-32	512	30.8%	69.2%	0.487

Three results are worth drawing out. BGE ships with an optional instruction prefix for queries, and adding it made retrieval *worse* here, so it is not used. Nomic-embed-text was tested because its 8192-token context and Matryoshka dimensions are attractive on paper, but our chunks are 220 words, so the long context is never exercised, and it scored below the smaller BGE model while also requiring `trust_remote_code`. On the image side, moving from 32-pixel to 16-pixel patches doubled top-1 accuracy, which is what the task needs: the held-out set is all waterfalls, so the encoder has to separate photographs of visually similar subjects. The larger ViT-L/14 did not help, scoring the same top-1 and lower MRR at three times the compute.

The margins are small in absolute terms. The text comparison turns on two questions out of 22 and the image comparison on four photographs out of 13, so these results indicate a direction rather than settling the question.

6 Retrieval workflow

6.1 Routing

One LLM call classifies the question as `structured`, `semantic`, `image` or `hybrid` and extracts a semantic sub-query plus any category or district the question restricts itself to. The class decides which retrievers run.

Query type	SQL	Text	Images
structured	✓		
semantic		✓	
image			✓
hybrid	✓	✓	✓

An uploaded photograph always forces image retrieval. If the routing call fails after it retries the query falls back to hybrid and runs everything. That is slower, but it never retrieves less than the question needs.

6.2 Structured retrieval

The model is given the schema and writes a `SELECT`, which is checked before it runs. Anything that does not start with `SELECT`, contains a second statement, or matches `INSERT`, `UPDATE`, `DELETE`, `DROP`, `ALTER`, `TRUNCATE`, `GRANT`, `REVOKE`, `CREATE` or `COPY` is rejected.

An early version had a bug here. The model wrote predicates like `province = 'Central Province'` against a column storing `'Central'`. The SQL was valid, ran without error and returned no rows, which from the user’s side looks the same as the attraction not existing. The fix was to read the distinct values of every low-cardinality column out of the database at start-up and put them in the prompt. A model writing SQL needs the data’s vocabulary as well as its schema.

6.3 Vector retrieval

Semantic search embeds the sub-query and takes the top k chunks. If the router identified a category or district, that becomes a ChromaDB metadata filter applied before the vector comparison, so “temples near Kandy” searches only Kandy instead of hoping the embedding encodes the district. Structured knowledge narrows the semantic search here. A filter that matches nothing falls back to an unfiltered search rather than returning an empty result.

Image search embeds either the uploaded photograph or the text sub-query against the same CLIP collection.

6.4 Context integration

This is where the two databases meet.

Candidates are ranked by attraction rather than by chunk, since vector search often returns several chunks and photographs belonging to the same place, and one well-documented site would otherwise fill the context window.

Every attraction found by either vector store is then looked up in PostgreSQL and its full row added to the context. A purely semantic query like “where can I see colonial-era fortifications beside the sea?” retrieves prose about Galle Fort, and this step gives the model its district, era, UNESCO status and coordinates, none of which appear in the retrieved text.

Weak image matches are dropped. CLIP similarities sit in a narrow band, so a fixed threshold does not work, but the gap below the best match does. A photograph of Sigiriya returns the right image at 1.00 while unrelated waterfalls still score around 0.85, and passing those on made the model describe attractions nobody asked about. Hits more than 0.10 below the best match are removed.

6.5 Generation

The context goes to Gemma 4 31B along with the retrieved photographs as image parts rather than filenames, so the generation is genuinely multimodal.

The system prompt tells the model to answer only from the context, cite passages as [S1], and reply that the knowledge base does not cover the question rather than filling the gap from general knowledge. A tourism system that invents an entrance fee is worse than one that admits it does not know, because the user cannot tell the difference.

7 Evaluation

7.1 Method

We wrote 36 evaluation questions, each labelled with its expected query type and the attractions a correct answer has to be grounded in: 8 structured, 16 semantic, 6 hybrid and 6 image. Four things were measured: router accuracy over all 36, text retrieval over the 22 semantic and hybrid questions (Recall@ k and MRR, ranked by attraction), image retrieval on the 13 held-out photographs, and an ablation.

An earlier version of the evaluation set had 12 questions, which left only six for text retrieval. At that size a single question moved Recall@1 by 17 points, so no comparison between retrieval strategies or encoders meant anything. The questions were written against passages in the corpus and deliberately worded differently from the source text, so they test retrieval rather than keyword overlap.

The ablation measures *context recall*: how often the expected attraction reaches the model under each retrieval strategy. We used that rather than scoring answer wording because it is objective. If the right attraction never reaches the model, no prompt will fix the answer.

7.2 Results

Metric	Result
Router accuracy	69.4% (25/36)
Text Recall@1	68.2%
Text Recall@3	90.9%
Text Recall@5	95.5%
Text MRR	0.792
Image top-1	61.5% (n=13)
Image top-3	69.2% (n=13)
Context recall, SQL only	38.9%
Context recall, semantic only	91.7%
Context recall, hybrid	97.2%

7.3 Discussion

The ablation is the main result, and it is clearest broken down by question type.

Question type	n	SQL only	Semantic only	Hybrid
structured	8	75%	75%	100%
semantic	16	12%	94%	94%
image	6	0%	100%	100%
hybrid	6	100%	100%	100%
all	36	38.9%	91.7%	97.2%

Retrieving from the relational store alone reaches the right attraction in 38.9% of cases, and it fails in a structured way: 12% on semantic questions and 0% on image questions, because no column encodes “has frescoes” or “looks like this photograph”. Combining the stores beats either alone, and on structured questions the margin is decisive, 100% against 75% for semantic search on its own.

The margin between semantic-only and hybrid is narrow, 91.7% against 97.2%, and that is worth stating plainly rather than presenting the combination as transformative. Semantic search does well on image questions because a request like “show me pictures of an ancient rock fortress” describes the attraction in words that match its text chunks, so the right attraction is found even without touching the image index. The image index still earns its place by answering questions posed *with* a photograph rather than about one, which the evaluation set cannot express as a text query.

An earlier version of this evaluation used 12 questions and reported 58.3%, 66.7% and 100% for the three strategies. Those figures are not comparable to the ones above. The evaluation set was rebalanced from three questions per type to 8 structured, 16 semantic, 6 hybrid and 6 image, so a strategy that cannot answer semantic questions is now penalised much more heavily. The change in the SQL-only figure is mostly this rebalancing, not a regression.

Only one question fails under the full pipeline. Q24 asks which waterfall was reduced by a dam built under the Laxapana project. The fact is present in the AmazingLanka text for Aberdeen Falls, but the chunk containing it does not rank in the top five.

Routing. At 69.4% the router is the weakest component. Its errors are mostly over-classification: five semantic and four image questions were routed as `hybrid`, which costs time and context budget but not correctness, since `hybrid` retrieves a superset. One hybrid question was routed as `structured`, which is the more serious kind of error, because a narrower strategy can lose information the question needed.

The same 36 questions were also run against `gemini-3.1-flash-lite`, which routed 83.3% correctly against Gemma’s 69.4%. Retrieval quality is identical between the two, since the encoders do not change, so the difference is entirely in classification. A stronger model for the routing call alone would be the cheapest available improvement, and there is no reason routing and answering have to use the same model.

We did not tune the router prompt against these questions. The evaluation set is the only measurement available, and tuning against it would leave nothing to evaluate on.

Text retrieval. Recall@3 of 90.9% and MRR of 0.792 mean the right attraction is usually at or near the top. Recall@1 of 68.2% is the figure with most room left, and reranking the retrieved chunks would be the obvious way to raise it.

Image retrieval. Top-1 of 61.5% is measured on the hardest available task: all 13 held-out photographs are waterfalls, so the encoder has to tell Aberdeen Falls from Galboda Ella rather than a beach from a temple. Top-3 of 69.2% matters more in practice, since the shortlist is what reaches the model.

A measurement error worth reporting. An early run put SQL-only context recall at 25%.

The ablation issues over a hundred pipeline runs in quick succession, the free-tier API rate-limited part of that burst, and the harness scored those failed calls as retrieval misses. We added backoff to the API client and separated an `api_error` flag from an empty result, so unresolved failures are excluded from the averages instead of counted against retrieval. 25% is plausible for SQL-only retrieval, so nothing about it looked wrong, and it would have been reported as a result.

7.4 Limitations

- `entrance_fee_lkr`, `swimming_safety`, `surf_season` and `accessibility` are NULL for all 42 rows. Neither source reports them reliably and they were not curated in time, so the system answers that it does not know, and the structured demonstration uses fewer columns than the schema supports.
- Kurundu Oya Falls has no photographs, because Commons search returned none usable, so it cannot be found through image retrieval.
- Semantic search has no per-attraction diversity cap. One query returned six chunks all belonging to Galle Fort.
- The SELECT-only guard is enforced in application code. The database user is not read-only.
- There is no reranking, which is the obvious way to raise Recall@1 from 68.2%.
- Thirty-six questions and thirteen images are still a small sample. The gap between semantic-only and hybrid rests on two questions.
- The `summary` column is deliberately excluded from the assembled context to avoid duplicating the lead paragraph. Where a fact appears only there and not in a retrieved chunk, the system answers that it does not know even though the database holds the answer.
- National parks and wildlife were dropped for time. Distinct species are much easier to tell apart visually than one waterfall from another, so their absence makes the image task harder than it would have been.

8 Implementation

Language	Python 3.12
Relational store	PostgreSQL 16 (Docker)
Vector store	ChromaDB (persistent, local)
Encoders	bge-base-en-v1.5, clip-ViT-B-16
Generation	Gemma 4 31B (gemma-4-31b-it)
Interface	Streamlit

<code>src/config.py</code>	tunable parameters
<code>src/schema.sql</code>	relational schema
<code>src/sources.py</code>	Wikipedia / AmazingLanka / Commons clients
<code>src/ingest.py</code>	web sources -> PostgreSQL
<code>src/embed.py</code>	PostgreSQL -> ChromaDB
<code>src/retrieval.py</code>	router and the three retrievers
<code>src/generate.py</code>	LLM prompting and answering
<code>src/pipeline.py</code>	the five stages
<code>src/evaluate.py</code>	evaluation harness
<code>app.py</code>	Streamlit interface

The interface shows the routing decision, the SQL that ran, the retrieved chunks with their similarity scores, the matched photographs and the exact context sent to the model. That makes it possible to check where an answer came from.

Ingest is destructive and reproducible. Each run re-applies the schema and rebuilds from the curated CSV plus the web sources, committing per attraction so a network failure does not lose earlier work. Installation and run instructions are in `README.md`.

Acknowledgements

- Wikipedia and Wikimedia Commons, for descriptive text, coordinates and all 164 photographs, under CC BY-SA. Per-image licence and author are stored in the `images` table.
- AmazingLanka (<https://amazinglanka.com/wp/>), for additional text on 26 attractions, used for non-commercial academic purposes with attribution. No images were taken from this source.
- Models: BAAI/bge-base-en-v1.5 (MIT), OpenAI CLIP via `sentence-transformers`, Google Gemma via the Gemini API.
- Libraries: PostgreSQL, ChromaDB, `sentence-transformers`, PyTorch, Streamlit, `psycopg`, `Pillow`, `requests`, `google-genai`.

AI-assisted development tools were used during implementation. The design decisions, evaluation method and analysis are our own.

References

1. Lewis, P. et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS.
2. Radford, A. et al. (2021). *Learning Transferable Visual Models From Natural Language Supervision*. ICML.
3. Xiao, S. et al. (2023). *C-Pack: Packed Resources For General Chinese Embeddings*. arXiv:2309.07597.
4. ChromaDB documentation. <https://docs.trychroma.com/>
5. PostgreSQL 16 documentation. <https://www.postgresql.org/docs/16/>
6. UNESCO World Heritage Centre, Sri Lanka. <https://whc.unesco.org/en/statesparties/1k>